Fakulta informačních technologií, Vysoké učení technické v Brně

Architektury výpočetních systémů (AVS 2025)
Projekt č. 2: Paralelizace s OpenMP

Marta Jaroš (martajaros@fit.vut.cz)

Termín odevzdání: 8. prosince 2025

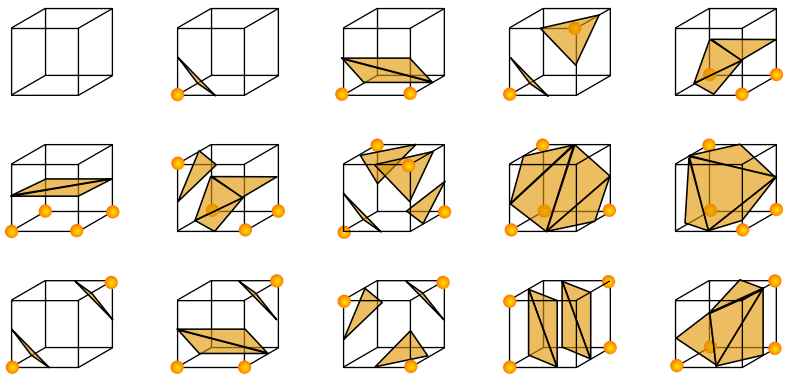
1. Úvod

Primárním cílem tohoto projektu je vyzkoušet si přechod od sekvenčního k paralelnímu algoritmu a jeho praktickou implementaci a optimalizaci. Druhotným cílem je pak osvojit si metriky používané při hodnocení efektivity paralelních algoritmů. Vaším prvním úkolem bude paralelizace naivního algoritmu pro rekonstrukci polygonálního povrchu ze skalárního pole ve 3D prostoru (tzv. "Marching Cubes") a vyhodnocení vlastností tohoto řešení. Druhým úkolem bude optimalizace tohoto řešení na úrovni algoritmu pomocí hierarchického dělení prostoru (tzv. "Octree") a rychlé eliminace prázdného prostoru. Veškeré paralelizace budou tvořeny pomocí OpenMP pragem pro *vícevláknovou aplikaci se sdílenou pamětí* — neboli sekce `#pragma omp parallel`. Cílem tedy není program explicitně vektorizovat.

Přístup k překladu a ladění je podobný, jako v prvním projektu. Jen namísto Intel Advisor budete využívat Intel VTune. Projekt lze přeložit a spustit prakticky kdekoliv (vyžaduje CMake a kompilátor), budeme se ale spoléhat na optimalizační reporty poskytované Intel kompilátorem. Referenčním strojem je výpočetní cluster Barbora s kompilátorem Intel (vizte A), na jehož výpočetním uzlu provádějte všechna měření do tohoto projektu. K ladění můžete použít i vlastní počítač, jestliže si nainstalujete nástroje od Intelu (jsou pro studenty zdarma). Pro prvotní ladění funkcionality algoritmu je možné využít i překladač GCC — algoritmus navrhnete a odladíte u sebe lokálně a až následně jej spustíte na clusteru.

2. Rekonstrukce povrchu pomocí Marching Cubes

Ačkoli implementace samotného jádra algoritmu "Marching Cubes"[1] není vaším úkolem, je důležité získat určitou představu o jeho fungování. Marching cubes je algoritmus využívaný v počítačové grafice pro rekonstrukci polygonové sítě (povrchu/isosurface[2]) ze skalárního pole[3]. Typicky je skalární pole získáváno jako 3D pole například jako data z CT nebo MRI. Pro účely projektu je však skalární pole vyhodnocováno za běhu a pouze v bodech, kde je nutné znát jeho hodnotu.
Samotný algoritmus je založen na rozdělení skalárního pole do krychlí o určité velikosti (udává rozlišení výsledné polygonové sítě). Při následném průchodu takto rozděleného pole je na základě hodnot ve vrcholech každé krychle určena konfigurace polygonů potřebná pro reprezentaci části povrchu procházejícího danou krychlí. Vhodná konfigurace polygonů je vybrána z $2^8 = 256$ možných konfigurací (obrázek 2.1) tak, že hodnoty ve vrcholech krychle jsou převedeny do binární podoby porovnáním s požadovanou hodnotou na povrchu (isosurface level) – těchto 8 binárních hodnot pak tvoří 8-bit ukazatel do tabulky možných konfigurací. Pro příklad uvažujme krychli, kde všechny její vrcholy mají hodnotu menší než požadovaná hodnota – dostáváme 8-bit index 0 (`00000000b`), který ukazuje na konfiguraci bez polygonů (celá krychle leží pod povrchem). Stejný výsledek získáme, pokud celá krychle leží nad povrchem – index 255 (`11111111b`).



Obrázek 2.1: 16 z 256 možných konfigurací polygonů

Konečně posledním krokem je lineární interpolace vygenerovaných vrcholů podél hrany krychle, tak aby výsledný vrchol ležel přesně na průsečíku mezi povrchem a příslušnou hranou krychle. Obrázek 2.1 ukazuje stav před interpolací a vrcholy tedy vždy leží ve středu hrany.
Algoritmus tedy může být rozdělen do tří pomyslných částí: rozdělení a průchod skalárním polem, výpočet hodnoty skalárního pole v daném bodě a vytvoření výstupních polygonů. Tento projekt je zaměřen na první dvě z těchto částí, které mohou být snadno paralelizovány pomocí OpenMP.

[1] `https://en.wikipedia.org/wiki/Marching_cubes`
[2] `https://en.wikipedia.org/wiki/Isosurface`
[3] `https://en.wikipedia.org/wiki/Scalar_field`

3. Struktura projektu, překlad a spuštění

Archiv zadání obsahuje:

```
PMC-xlogin00.txt
1_2.txt
2_1.txt
3_4.txt
4_1.txt
4_2.txt
4_3.txt
data/
scripts/
```

Soubor `PMC-xlogin00.txt` obsahuje otázky, které je třeba zodpovědět v průběhu řešení projektu. Na některé otázky ze souboru `PMC-xlogin00.txt` budete odpovídat v souborech `1_2.txt`, `2_1.txt`, `3_4.txt`, `4_1.txt`, `4_2.txt` nebo `4_3.txt`. Řiďte se instrukcemi v `PMC-xlogin00.txt`. Složka `data` obsahuje data pro testování a měření výkonnosti vašeho řešení. Složka `scripts` obsahuje skripty pro generování vstupních dat a grafů škálování. A dále obahuje zdrojový kód projektu. Navíc jsou k dispozici dva skripty pro cluster — `vtune.sl`, který provede analýzu a `evaluate.sl`, který vaši implementaci ohodnotí pro různá nastavení.
Zdrojový kód projektu je strukturován do několika tříd, které kopírují základní části algoritmu marching cubes. Hlavní část algoritmu je rozdělena mezi virtuální třídu `BaseMeshBuilder` a z ní dědící třídu `RefMeshBuilder`. Třída `BaseMeshBuilder` obsahuje kód samotného generování trojúhelníků, ale způsob průchodu skalárním polem a výpočet hodnoty v bodě je delegován virtuálními metodami (pure virtual) do dceřiné třídy `RefMeshBuilder`. Pro načítání a předzpracování vstupních dat slouží třída `ParametricScalarField`, která vstupní data zpřístupňuje v podobě kolekce bodů v 3D prostoru. A konečně prázdné třídy `LoopMeshBuilder` a `TreeMeshBuilder` dědící z `BaseMeshBuilder` jsou připraveny pro vaše paralelní implementace (`RefMeshBuilder` je tedy vzorem pro vaše řešení). Zdrojový kód je rozdělen do několika souborů:

- `main.cpp`: Obsahuje vstupní bod programu, zpracování parametrů (knihovna cxxopts[4]), nastavení počtu OpenMP vláken a výběr požadované implementace (referenční, paralelní smyčka, paralelní strom).
- `common/vector_helpers.h`: Obsahuje pomocnou třídu pro reprezentaci bodu v 3D prostoru (`Vec3_t`) a nástroje pro přesné měření realného času pomocí C++11.
- `common/base_mesh_builder.*`: Bázová třída pro vaše paralelní implementace Marching cubes.
- `common/parametric_scalar_field.*`: Třída reprezentující skalární pole jako množinu bodů v 3D prostoru.

[4] `https://github.com/jarro2783/cxxopts`

- `common/ref_mesh_builder.*`: Referenční implementace třídy `BaseMeshBuilder`. Její metody `marchCubes`, `evaluateFieldAt` a `emitTriangle` implementují průchod polem, vyhodnocení hodnoty pole v bodě a uložení jednoho vygenerovaného trojúhelníku. Metoda `getTrianglesArray` naopak vrací ukazatel na pole obsahující všechny vygenerované trojúhelníky.

- `parallel_builder/loop_mesh_builder.*`: Bude obsahovat vaši implementaci (obdobu `RefMeshBuilder`) pomocí paralelních smyček OpenMP.

- `parallel_builder/tree_mesh_builder.*`: Bude obsahovat vaši implementaci (obdobu `RefMeshBuilder`) optimalizovanou pomocí paralelního stromového průchodu a OpenMP tasků.

Pro přeložení projektu je možné použít `CMake` následujícím způsobem:

```
~$ cd Assignment
~/Assignment$ mkdir build && cd build
ml CMake/3.27.6-GCCcore-13.2.0 intel-compilers/2024.2.0
~/Assignment/build$ CC=icx CXX=icpx cmake ..
~/Assignment$ make -j
```

Výstupem překladu by měl být jediný spustitelný soubor `PMC` (Parallel Marching Cubes), jehož výstupem je:

```
~/Assignment/build$ ./PMC

AVS Assignment 2 - Parallel Marching Cubes
Usage:
  AVS: Marching cubes [OPTION...] INPUT <OUTPUT>

  -l, --level arg    Iso surface value (default: 0.15)
  -g, --grid arg     Grid size (default: 64)
  -b, --builder arg  Builder name [ref, loop, tree] (default: ref)
  -t, --threads arg  Number of OpenMP threads (default: 0)
      --batch        Run in silent/batch mode
  -h, --help         Print help

Missing required arguments: "INPUT"
```

Argumenty programu mají následující významy:

- `INPUT` soubor
  Vstupní soubor obsahující body, které definují skalární pole.

- `OUTPUT` soubor
  Výstupní soubor pro vytvořený model ve formátu "Wavefront *.obj". Tento parametr lze vynechat – výstup je pak zahozen (vhodné pro měření rychlosti).

- `--level l`
  Udává hodnotu "isosurface", tedy vzdálenost k nejbližšímu bodu ve vstupním souboru, na které leží hledaný povrch. Ve zdrojovém kódu lze tuto honotu přečíst z proměnné `mIsoLevel` ve třídě `BaseMeshBuilder`, nebo voláním metody `getIsoLevel` na objektu třídy `ParametricScalarField`.

- `--grid g`
  Udává délku hrany diskretizační mřížky a s ní rozlišení diskretizace a celkový počet "Marching cubes" ($N = g^3$). Hodnotu tohoto parametru lze získat v proměnné `mGridSize` ve třídě `BaseMeshBuilder`.

- `--builder`
  Umožňuje zvolit konkrétní implementaci algoritmu "Marching cubes". Možnosti jsou:
  - `ref` Referenční implementace v souborech `common/ref_mesh_builder.*`
  - `loop` Vaše implementace pomocí paralelní smyčky (vizte sekci 5.1)
  - `tree` Vaše implementace pomocí paralelního průchodu stromem (vizte sekci 5.2)

- `--threads N`
  Maximální počet OpenMP vláken (pokud není hodnota specifikována použije se nastavení prostředí). Nastavení je realizováno voláním funkce `omp_set_num_threads` z knihovny OpenMP.

- `--batch`
  Přepnutí do režimu minimálních výpisů ve formátu CSV, vhodné pro vytváření tabulek a grafů.

Spuštění programu s některým ze vstupních souborů ze složky `data` vygeneruje příslušný polygonální povrch:

```
~/Assignment/build$ ./PMC ../data/bun_zipper_res3.pts bun_zipper_res3.obj

======================== Marching Cubes Mesh Builder ==========================
Mesh Builder:        Reference
Input Field File:    ../data/bun_zipper_res3.pts
Output Mesh File:    bun_zipper_res3.obj
Grid Size:           64
Iso Level:           0.15
Field Elements:      1889
============================ Building the Mesh ================================
Number of Threads:   36
Elapsed Time:        1180 ms
Mesh Triangle Count: 40288
========================== Writting Output File ===============================
Output File Size:    5129 KB
=================================== DONE ======================================
```

Výstup programu ukazuje, že byla použita referenční implementace na vstupním souboru `../data/bun_zipper_res3.pts` a výstup byl uložen do souboru `bun_zipper_res3.obj`. Hrana mřížky byla 64, "iso-level" měl hodnotu 0.15 a vstupní soubor obsahoval 1889 bodů. Výstupní model byl vygenerován za 1180 ms, obsahuje 40288 trojúhelníků a soubor má velikost 5129 KB.

Formát vstupních a výstupních souborů

Vstupní soubory obsahují sekvenci bodů v 3D prostoru:

```
../data/bun_zipper_res3.pts

p -0.312216 1.263040 0.051492
p -0.446774 1.312040 0.057048
p -0.683011 1.448280 0.413688
...
```

Výstupem je polygonální síť ve formátu "Wavefront *.obj", soubor tedy obsahuje seznam vrcholů a seznam trojic indexů spojující vrcholy do trojúhelníků:

```
bun_zipper_res3.obj

v 9.88281631 8.1788826 0.339680344
v 9.86799812 8.1788826 0.340786725
v 9.88281631 8.17489433 0.340786755
...
f 1 2 3
f 4 5 6
f 7 8 9
...
```

Soubor tohoto formátu lze vizualizovat většinou nástrojů pro práci s 3D grafikou — ve Windows Microsoft 3D Viewer, online nástroj `https://3dviewer.net/`, nebo open-source Blender[5]: File → Import → Wavefront (*.obj). Je možné vizualizovat pouze výstup algoritmu.
Pro ladění byl vytvořen skript `scripts/check_output.py <A.obj> <B.obj>`, který otestuje, zda jsou výstupy shodné a jaký je jejich rozdíl. Pro rychlejší ladění byl vytvořen i skript `scripts/compare.sh`, který na dvou velikostech zkusí, za obě dvě implementace pro `bun_zipper_res3.pts` dávají stejné výsledky, jako referenční algoritmus. Skript lze spustit jako:

```
cd build_evaluate
ml matplotlib/3.5.2-foss-2022a
ml CMake/3.27.6-GCCcore-13.2.0 intel-compilers/2024.2.0
bash ../scripts/compare.sh
```

4. Výstup a hodnocení projektu

Výstupem projektu bude archiv `xlogin00.zip` obsahující vámi modifikované soubory (jiné soubory, než ty zmíněné níže, nemodifikujte a tedy ani neodevzdávejte), textové soubory s otázkami a odpověďmi (celkem 11 souborů) a tři grafy škálování (vizte sekci 5.3). Do patřičných textových souborů uveďte výsledky měření a zodpovězte požadované otázky. V každém souboru (který jste změnili) nezapomeňte uvést svůj login! Výjimkou jsou soubory `1_2.txt`, `2_1.txt`, `3_4.txt`, `4_1.txt`, `4_2.txt` a `4_3.txt`. Hodnotit se bude jak funkčnost, správnost a rychlost implementace, tak textový komentář – ten by měl

[5] `https://www.blender.org/`

stručně, věcně ale dostatečně popisovat výsledky měření a odpovídat na otázky uvedené v zadání. Hodnocení je uvedené u jednotlivých částí zadání a dohromady tvoří 15 bodů. Projekt odevzdejte v uvedeném termínu do informačního systému.

Shrnutí obsahu odevzdávaného archivu:

- `parallel_builder/loop_mesh_builder.{h/cpp}`
  Vaše implementace pomocí paralelních smyček – sekce 5.1
- `parallel_builder/tree_mesh_builder.{h/cpp}`
  Vaše implementace pomocí paralelního stromu – sekce 5.2
- `PMC-xlogin00.txt` (přejmenujte dle vašeho loginu)
  Odpovědi na uvedené otázky.
- `1_2.txt`
  Měření k otázce 1.2 z `PMC-xlogin00.txt`.
- `2_1.txt`
  Měření k otázce 2.1 z `PMC-xlogin00.txt`.
- `3_4.txt`
  Odpověd na otázku 3.4 z `PMC-xlogin00.txt`.
- `4_1.txt`
  Odpověd na otázku 4.1 z `PMC-xlogin00.txt`.
- `4_2.txt`
  Odpověd na otázku 4.2 z `PMC-xlogin00.txt`.
- `4_3.txt`
  Odpověd na otázku 4.3 z `PMC-xlogin00.txt`.
- `input_scaling_strong.png`, `input_scaling_weak.png` a `grid_scaling.png`
  Grafy silného a slabého škálování vzhledem ke vstupu a graf škálování vzhledem k velikosti mřížky.

5. Paralelní Marching Cubes (15 bodů)

Cílem projektu je vytvořit trojici tříd implementující hlavní části algoritmu Marching cubes, tak aby bylo možné využít více-jádrové CPU. Následující výpis ukazuje signaturu těchto tříd a referenční (sekvenční) implementaci je možné nalézt v souboru `common/ref_mesh_builder.cpp`.

```
class MyMeshBuilder : public BaseMeshBuilder
{
public:
    MyMeshBuilder(unsigned gridEdgeSize);

protected:
    unsigned marchCubes(const ParametricScalarField &field);
    float evaluateFieldAt(const Vec3_t<float> &pos, const  ParametricScalarField &field);
    void emitTriangle(const Triangle_t &triangle);
    const Triangle_t *getTrianglesArray() const;
};
```

Každá z těchto tříd musí implementovat následující virtuální metody:

- `unsigned marchCubes(const ParametricScalarField & field)`
  Implementace průchodu mřížkou, tedy každou pozicí v 3D prostoru, kde je třeba vygenerovat polygony. Pro samotné vyhodnocení/vygenerování polygonů na dané pozici lze zavolat metodu `unsigned buildCube(/* position */, /* field */)`, která je definovaná ve třídě `BaseMeshBuilder` a očekává pozici krychle v mřížce a příslušné pole "field" a vrací počet vygenerovaných trojúhelníků.
  Metoda `marchCubes` je volána metodou `BaseMeshBuilder::buildMesh`, která očekává, že návratová hodnota `marchCubes` bude celkový počet vygenerovaných polygonů.

- `float evaluateFieldAt(const Vec3_t<float> &pos, const /* ... */ &field)`
  Implementuje výpočet hodnoty skalárního pole `field` v bodě `pos` a vrací vypočtenou hodnotu. Tato hodnota je dána výrazem 5.1, který je vlastně pouhým nalezením vzdálenosti od bodu `pos` k nejbližšímu bodu s v poli `field`.

  $$\min_{s \in F} \sqrt{\sum\nolimits_{\xi \in \{x,y,z\}} (s_\xi - p_\xi)^2} \tag{5.1}$$

  Kde F je pole `field`, s je jeden z bodů definující pole F, $i \in \{x, y, z\}$ označuje dimenze a p je bod `pos`.

  POZOR: Tato metoda je volána z `BaseMeshBuilder::buildCube`!

- `void emitTriangle(const Triangle_t &triangle)`
  Metoda je volána po vygenerování každého trojúhelníku a jejím úkolem je uložit daný trojúhelník do vhodné datové struktury (v referenční implementaci postačuje C++ vektor). Metoda může být volána několikrát v rámci jednoho volání `BaseMeshBuilder:: buildCube`, což vyžaduje zvýšenou pozornost.

  POZOR: Tato metoda je volána z `BaseMeshBuilder::buildCube`!

- `const Triangle_t *getTrianglesArray() const`
  Metoda vrací ukazatel na pole trojúhelníků (tj. oblast paměti, kde jsou trojúhelníky těsně za sebou) a je volána pouze 1× z `BaseMeshBuilder::buildMesh` těsně před uložením polygonů do výstupního souboru. Pořadí trojúhelníků v tomto poli je nedefinované.

Uspořádání prostoru

Pro práci s již implementovanými metodami jako `buildCube` je důležité korektní předávání souřadnic jako pozice krychle. Kód pracuje pouze v kladném oktantu, všechny souřadnice tedy leží v intervalu $\langle 0, S_{i\in\{x,y,z\}}\rangle$, kde S_i je délka hrany osově zarovnaného kvádru kolem vstupních dat. Vstupní data jsou po načtení do tohoto prostoru mapovány jednoduchým posunutím.
Rozlišení mřížky (tedy velikost jedné "Marching cube") je vypočtena jako

$$\texttt{mGridResolution} = \frac{\max_{i\in\{x,y,z\}} S_i}{\texttt{mGridSize}}. \tag{5.2}$$

Tím vzniká prostor celočíselných indexů $0, 1, \ldots, \frac{\texttt{mGridSize}}{\texttt{mGridResolution}}$ v každé dimenzi. Metoda `buildCube` očekává souřadnice krychle právě v těchto celočíselných indexech. Naproti tomu metoda `evaluateFieldAt` očekává pozici v původním spojitém prostoru! Výstupní polygony jsou taktéž v původním spojitém prostoru. **Při řešení předpokládejte pouze velikosti mřížek v mocninách 2 (`mGridSize` $\in 2^i$)** – což znatelně zjednodušuje stromovou implementaci (vizte dále).

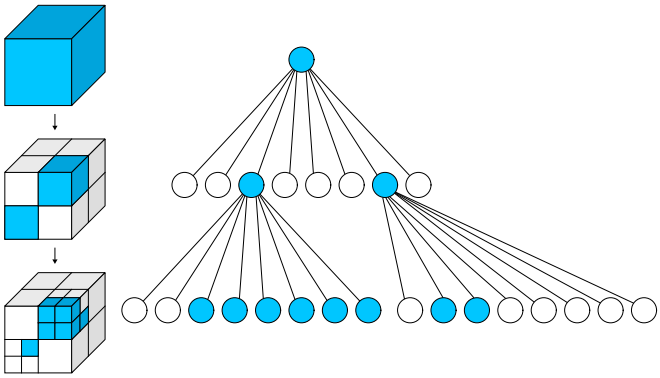
5.1. Paralelizace původního řešení (4 bodů)

Prvním úkolem je tedy vytvořit paralelní implementaci třídy `LoopMeshBuilder` v souborech `parallel_builder/loop_mesh_builder.{h/cpp}`. Jako výchozí kód můžete použít referenční implementaci v `common/ref_mesh_builder.{h/cpp}`, která obsahuje dvě pro paralelizaci kandidátní smyčky: průchod mřížkou v `marchCubes` a výpočet minimální vzdálenosti v `evaluateFieldAt`. Pro paralelizaci použijte vhodné prostředky OpenMP. V rámci úlohy vyzkoušejte paralelizaci obou smyček (nezávisle) a jako výsledné řešení odevzdejte to, které bude vykazovat lepší výsledky. Pokud se pokusíte paralelizovat obě smyčky zároveň, nezapomeňte, že půjde o vnořený paralelismus (`evaluateFieldAt` je efektivně volána ze smyčky procházející mřížkou).
Při paralelizaci smyček také ověřte vliv různých možností rozdělení práce mezi vlákna (OpenMP static/dynamic/guided sheduling) a výsledná zjištění stručně popište v textovém souboru s odpověďmi.

5.2. Paralelní průchod stromem (6 bodů)

Druhým úkolem je vytvořit optimalizovanou paralelní implementaci třídy `TreeMeshBuilder` v souborech `parallel_builder/tree_mesh_builder.{h/cpp}`. V tomto případě je však cílem nejen paralelizace, ale také optimalizace samotného algoritmu průchodu mřížkou.

Vzhledem k tomu, že hledaný povrch se nachází ve specifikované vzdálenosti od nejbližšího bodu s (výraz 5.1 v kombinaci s parametrem programu) lze relativně snadno určit oblast mřížky, kde není možné, aby jí procházel hledaný povrch. To umožňuje optimalizovat průchod mřížkou tak, že se tyto oblasti vyloučí a nebude se pro ně volat metoda `buildCube`.
Typickým přístupem k řešení tohoto problému je hierarchická dekompozice prostoru. V našem případě použijeme stromovou dekompozici (Octree decomposition[6]), kde se prostor dělí na každé úrovni na 8 potomků (vizte obrázek 5.1).



Obrázek 5.1: Octree dekompozice, kde modře vyznačenými bloky prochází hledaný povrch.

Průchod takovýmto stromem je pak následující:

- Aktuální blok (na začátku celá mřížka) je rozdělena do 8 potomků (bloky o poloviční délce hrany).
- Pro každého potomka ověříme, zda je možné, aby jeho podprostorem procházel hledaný povrch (isosurface). Pro účely tohoto projektu lze tento test snadno aproximovat pomocí opsané koule kolem aktuálního bloku (povrch je totiž také definovaný množinou bodů a vzdáleností – tedy množinou koulí na různých pozicích). Podmínka prázdnosti bloku je tedy

$$F(p) > l + \frac{\sqrt{3}}{2}a \tag{5.3}$$

  Kde $F(p)$ je hodnota pole (tedy 5.1) ve středu bloku, l je aktuální "iso-level" a a je délka hrany bloku. Při implementaci si dejte pozor na celočíselné operace (např. `double b = a / 2` je stále celé číslo, pokud `a` je integer).
- Konečně, každý neprázdný potomek je rozdělen na dalších 8 potomků až do předem specifikované hloubky (tzv. "cut-off"), nebo velikosti hrany a (která je zmenšena na polovinu s každým zanořením).

[6] `https://en.wikipedia.org/wiki/Octree`

- Na nejnižší úrovni jsou pak voláním `BaseMeshBuilder::buildCube` (jako v předchozím případě) vygenerovány samotné polygony pro všechny krychle náležející do daného podprostoru.

Je zřejmé, že pro tento rekurzivní proces jsou vhodným mechanismem pro paralelizaci OpenMP tasky a jejich využití je v tomto případě tedy povinné! Nezapomeňte také, že metoda `marchCubes` vrací počet vygenerovaných trojúhelníků. Jelikož tento počet bude známý až po zanoření na poslední úroveň, je vhodné použít synchronizaci tasků (rodič čeká na potomky) a případně atomické operace pro sesbírání počtu od každého potomka. Konečně, pro výpočet "fyzické" velikosti bloků stromu a pozice jejich středů je možno použít třídní proměnnou `mGridResolution` (deklarováno v `BaseMeshBuilder`). Proměnná `mIsoLevel` obsahuje aktuální "iso-level" potřebný pro vyhodnocení prázdnosti bloku.

5.3. Vyhodnocení škálování jednotlivých řešení (5 bodů)

Pro všechny varianty algoritmu vytvořte grafy silného a slabého škálování vzhledem k počtu bodů ve vstupním souboru a počtu vláken. Vytvořte také společný graf škálování vzhledem k velikosti mřížky pro hranu délky 8 – 512 bodů při použití maximálního počtu vláken.
Pro zjednodušení byl vytvořen skript pro Slurm `evaluate.sl`, který smaže původní výsledky a vytvoří data i grafy (do adresáře projektu). Vzhledem k náročnosti aplikace se tento skript musí pouštět v dávkovém režimu (přes `sbatch`).
Dále budete měřit využití jednotlivých jader pro celý dvouprocesorový systém (36 jader) a pro polovinu (18 jader). Měření automaticky provede Slurm skript `vtune.sl`, který pro jednotlivé konfigurace vytvoří adresář `build_vtune/vtune-{builder}-{threads}`. Vy potom v nástroji Intel VTune (který můžete pustit lokálně, v CVT, nebo i na uzlu (login uzel nemusí funguvat kvůli paměťovým nárokům) vyhodnotíte, jak byla využita jednotlivá jádra. V nástroji VTune stačí otevřít soubor (results, ne projekt!) `build_vtune/-vtune-{builder}-{threads}/vtune-{builder}-{threads}.vtune`.
Dosažené výsledky stručně zhodnoťte (a pokud možno vysvětlete) v textovém souboru s odpověďmi na otázky. Bez funkční implementace vám však za odpovědi na příslušné otázky nebudou uděleny žádné body!
Podobně jako projekt 1, i tento projekt bude hodnocen z hlediska správnosti výsledků, ale také rychlosti vašeho řešení vůči referenční verzi (NEJDE o soutěž mezi vámi)!

A. Superpočítač Barbora

Superpočítač Barbora umístěný na VŠB v Ostravě je složen z celkem 192 uzlů, každý uzel je osazen dvěma 18jádrovými procesory Intel Cascade Lake 6240 a 192 GB RAM. Tyto procesory podporují tedy vektorové instrukce AVX-512. Pro připojení na superpočítač Barbora je potřeba mít vytvořený účet, se kterým je možné se připojit na jeden z dvojice tzv. čelních (login) uzlů – `barbora.it4i.cz` (round-robin DNS). Postup získání přístupu byl popsaný v prvním projektu. Stejně tak připojení ke grafickému prostředí. Nezapomeňte na závěr smazat běžící instance VNC.

Po připojení ke vzdálené ploše můžete otevřít emulátor terminálu (menu Aplikace → Systémové nástroje → Terminál) a vytvořit interaktivní úlohu následujícím způsobem:

```
[user@login1.barbora ~]$ sbatch vtune.sl
[user@login1.barbora ~]$ squeue --me
# pockat, az uloha dobehne
[user@login1.barbora ~]$ ml VTune
[user@login1.barbora ~]$ vtune-gui
# pokud uloha skonci se segfault, je nutne VTune pustit na vypocetnim uzlu
# (použijte salloc pro získání interaktivní úlohy)

# nebo spustit interaktivni ulohu na uzlu
[user@login1.barbora ~]$ salloc -A ATR-25-7 -p qcpu_exp -N 1 --comment 'use:vtune=2022.2.0' --x11
salloc: Pending job allocation 68455
salloc: job 68455 queued and waiting for resources
...
salloc: job 68455 ready
[user@cn49.barbora ~]$
```

Příkaz `sbatch` zadá požadavek na spuštění úlohy do fronty; jakmile bude v systému dostatek volných uzlů, dojde ke spuštění úlohy. Parametr `-A` určuje projekt, v rámci kterého máme alokované výpočetní hodiny (neměnit), `-p` určuje frontu, do které bude úloha zařazena (používejte qcpu_exp nebo qcpu), parametr `-N` určuje zdroje, které budou úloze přiděleny (počet uzlů) a další možnosti (v našem případě načtení modulu pro profiler). Interaktivní úlohu pak obdobně pomocí příkazu `salloc`. Pomocí `--x11` tunelujeme XWindow system. Více o spouštění úloh na superpočítačích IT4I naleznete v dokumentaci[7].
Nyní jste již v terminálu připojeni k výpočetnímu uzlu (vizte nový hostname v terminálu) s tím, že by jste měli být schopní spustit grafickou aplikaci:

```
[user@cn49.barbora ~]$ ml VTune
[user@cn49.barbora ~]$ vtune-gui
```

Software na superpočítači je dostupný pomocí tzv. *modulů*. Práci s nimi zajišťuje příkaz `ml` (*module load*). Tento příkaz bez parametrů vypíše aktuálně načtené moduly, a všechny moduly specifikované jako parametry se pokusí načíst. Příkaz `ml purge` je všechny odstraní.
Moduly je potřeba načíst po každém přihlášení nebo získání výpočetního uzlu (jsou implementovány proměnnými prostředí). V tomto projektu budou pro překlad (a spouštění) potřeba moduly `intel-compilers` (kompilátor), `CMake` (překladový systém) a pro profilování modul `VTune` (jak bylo již demonstrováno výše). **Jestliže jej potřebujete, modul profileru načtěte jako první!** Závisí na starších knihovnách, které mohou být přepsány novějšími po načtení kompilátoru. Pro profilování kvůli knihovnám musíte mít načtený i modul `intel-compilers`, jinak se vaše přeložená aplikace nespustí. Pro vykreslení výsledků budete potřebovat Python3 s knihovnami NumPy a Matplotlib `matplotlib`.
Výstupy ze Slurm skriptů naleznete v souborech `AVS.*.{oe}{0-9}*`.

[7] https://docs.it4i.cz/en/docs/general/run-jobs/job-sub-exec/job-submission-and-execution